

Anatomía de un skill · Scripts reales Bash y Python · allowed-tools · Seguridad en skills · ClawHub y publicación · Ciclo completo

5

GUÍA 2 — MÓDULO 5

Skills Avanzados

Construí, auditá y publicá skills de producción — con conciencia de los riesgos reales

I — Anatomía completa de un skill de producción

Un skill de OpenClaw es una carpeta que contiene un archivo SKILL.md y, opcionalmente, scripts de soporte. La documentación oficial del ClawHub y el repositorio de skills del equipo de OpenClaw definen una estructura estándar que conviene respetar para que el skill funcione bien en producción y pueda publicarse sin problemas.

Estructura de archivos

```
$ mi-skill/
SKILL.md          # obligatorio — instrucciones para el modelo
scripts/
  accion.sh       # scripts Bash que el skill puede invocar
  procesar.py     # scripts Python opcionales
references/
  referencia.md   # documentación de apoyo cargada bajo demanda
README.md        # descripción para humanos (no la lee el modelo)
```

SKILL.md completo — frontmatter y cuerpo

```
---
$ nombre: monitor-uptime
$ descripcion: >
  Verificá si un sitio web está activo y respondiendo correctamente.
  Usá este skill cuando el usuario pregunte por el estado de un servidor,
  quiera revisar si una URL responde, o necesite chequear códigos HTTP.
  Funciona con cualquier URL pública. No requiere credenciales.
$ version: 1.0.0
$ metadata:
  openclaw:
    requiere:
      bins: ["curl"]
      env: []
      herramientas-permitidas: ["bash"]
---

# Monitor de Uptime
$ Verificación rápida de disponibilidad de sitios web.

## Chequeo básico
$ ```bash
$ curl -s -o /dev/null -w "%{http_code} | %{time_total}s" https://sitio.com
$ ```
$ Respuesta esperada: código 200 y tiempo menor a 3 segundos.

## Reglas
- Si el código es 200: el sitio está activo.
```

- Si es 5xx: el servidor tiene un error. Notificar.
- Si hay timeout (más de 30 segundos): el sitio no responde. Notificar.
- Nunca reintentar más de 2 veces si el primer chequeo falla.

→ El frontmatter es la primera línea de defensa

Los campos metadata.openclaw.requiere y herramientas-permitidas le dicen a OpenClaw exactamente qué necesita el skill para funcionar. Si el frontmatter dice que el skill solo necesita bash, y en el cuerpo hay instrucciones para usar browser o ejecutar código Python, eso es una señal de alarma — el skill está pidiendo más de lo que declara.

Al auditar un skill de terceros, lo primero que hay que revisar es si los permisos declarados en el frontmatter son coherentes con las instrucciones del cuerpo. Un skill de clima no necesita acceso a shell.execute. Si lo tiene declarado, no instalar.

Tipos de skills — qué puede hacer cada uno

II — Los cuatro tipos de skills

La documentación de OpenClaw y la comunidad de ClawHub organizan los skills en cuatro categorías según su mecanismo de ejecución. La categoría determina la complejidad de construcción, los permisos necesarios y el nivel de riesgo inherente.

Tipo	Cómo funciona	Permisos necesarios	Riesgo
Solo instrucciones	El modelo sigue instrucciones de texto. No ejecuta código. Ejemplo: un skill de prompt engineering, un skill de escritura formal.	Ninguno	Muy bajo
Skill con Bash	El modelo ejecuta comandos de shell usando la herramienta bash. Puede llamar a APIs, leer archivos, correr scripts.	bash	Medio — depende del script
Skill con Python	El modelo ejecuta scripts Python para tareas más complejas: procesamiento de datos, requests HTTP con lógica, manejo de archivos JSON.	bash + python3 instalado	Medio-alto
Skill con browser	El modelo controla un browser headless para navegar, hacer scraping o interactuar con sitios que no tienen API.	browser	Alto — acceso a sesiones web

III — Construir un skill con Bash

El Bash skill es el más común en producción. El modelo recibe instrucciones, decide cuándo ejecutar el script, lo llama con los parámetros correctos e interpreta la salida. La lógica compleja vive en el script; el SKILL.md le dice al modelo cómo y cuándo usarlo.

Ejemplo real — skill de backup automatizado

```
# scripts/backup.sh — el script que hace el trabajo
#!/bin/bash

$ ORIGEN="$1"
$ DESTINO="$2"
$ TIMESTAMP=$(date +%Y%m%d_%H%M%S)
$ ARCHIVO_BACKUP="${DESTINO}/backup_${TIMESTAMP}.tar.gz"

# Verificar que el directorio origen existe
$ if [ ! -d "$ORIGEN" ]; then
  echo "ERROR: El directorio $ORIGEN no existe"
  exit 1
$ fi

# Crear backup comprimido
$ tar -czf "$ARCHIVO_BACKUP" "$ORIGEN" 2>&1

$ if [ $? -eq 0 ]; then
  TAMANIO=$(du -sh "$ARCHIVO_BACKUP" | cut -f1)
  echo "OK: Backup creado en $ARCHIVO_BACKUP ($TAMANIO)"
$ else
  echo "ERROR: Falló la creación del backup"
  exit 1
$ fi
```

```
# SKILL.md — las instrucciones para el modelo
---
$ nombre: cron-backup
$ descripcion: >
  Creá backups comprimidos de directorios. Usá cuando el usuario quiera
  hacer backup de una carpeta, programar respaldo automático o verificar
  el último backup existente.
$ version: 1.0.0
$ metadata:
  openclaw:
    requiere:
      bins: ["bash", "tar"]
      herramientas-permitidas: ["bash"]
---

# Backup de directorios

## Crear un backup ahora
$ ``bash
$ ./scripts/backup.sh /ruta/origen /ruta/destino
```

```
$ `` `

## Interpretar la salida
- "OK: Backup creado en..." – éxito. Informar la ruta y el tamaño al usuario.
- "ERROR: El directorio no existe" – el origen es incorrecto. Pedir al usuario la ruta correcta.
- "ERROR: Falló la creación del backup" – problema de permisos o espacio. Verificar ambos.

## Reglas
- Siempre confirmar con el usuario la ruta de origen antes de ejecutar.
- Si no se especifica destino, usar ~/backups/ como valor por defecto.
```

IV — Construir un skill con Python

Python permite lógica más sofisticada que Bash: manejo de JSON, requests HTTP con headers complejos, procesamiento de datos, y mejor manejo de errores. El patrón es el mismo: el script hace el trabajo, el SKILL.md le dice al modelo cómo usarlo.

Ejemplo real — skill que consulta una API y procesa la respuesta

```
# scripts/consultar_api.py
$ import sys
$ import json
$ import urllib.request
$ import urllib.error
$ import os

$ def consultar(endpoint, recurso_id=None):
    token = os.environ.get("MI_API_TOKEN")
    if not token:
        print(json.dumps({"error": "MI_API_TOKEN no configurado"}))
        sys.exit(1)

    url = f"https://api.servicio.com/v1/{endpoint}"
    if recurso_id:
        url += f"/{recurso_id}"

    req = urllib.request.Request(url)
    req.add_header("Authorization", f"Bearer {token}")
    req.add_header("Content-Type", "application/json")

    try:
        with urllib.request.urlopen(req, timeout=30) as resp:
            datos = json.loads(resp.read())
            print(json.dumps(datos, ensure_ascii=False, indent=2))
    except urllib.error.HTTPError as e:
        print(json.dumps({"error": f"HTTP {e.code}", "detalle": str(e)}))
        sys.exit(1)
    except Exception as e:
        print(json.dumps({"error": str(e)}))
        sys.exit(1)

$ if __name__ == "__main__":
    endpoint = sys.argv[1] if len(sys.argv) > 1 else "recursos"
    rid = sys.argv[2] if len(sys.argv) > 2 else None
    consultar(endpoint, rid)
```

```
# SKILL.md correspondiente
---
$ nombre: mi-api-cliente
$ descripcion: >
    Consultá la API de MiServicio para obtener recursos y estado.
```

```

    Usá cuando el usuario pida información de la plataforma.
$ metadata:
  openclaw:
    requiere:
      bins: ["python3"]
      env: ["MI_API_TOKEN"]
      herramientas-permitidas: ["bash"]
---

## Consultar todos los recursos
$ ```bash
$ python3 scripts/consultar_api.py recursos
$ ```

## Consultar un recurso específico
$ ```bash
$ python3 scripts/consultar_api.py recursos ID_DEL_RECORSO
$ ```

## Interpretar la salida
- JSON con datos: mostrar los campos relevantes al usuario en lenguaje natural.
- {"error": "MI_API_TOKEN no configurado"}: avisar al usuario que falta configurar la variable.
- {"error": "HTTP 401"}: el token es inválido. Pedir al usuario que lo revise.
- {"error": "HTTP 429"}: esperar 60 segundos y reintentar una vez.

```

→ Por qué usar urllib en lugar de requests

El módulo requests es más cómodo, pero requiere instalación externa. urllib viene incluido en Python 3 sin instalación adicional. Para scripts de skills que van a correr en sistemas de otros usuarios, usar la librería estándar es una decisión de robustez: el skill funciona en cualquier Python 3 sin pasos adicionales de instalación.

V — Seguridad en skills: riesgos reales documentados

El ecosistema de skills de ClawHub tuvo una crisis de seguridad documentada en enero y febrero de 2026. Entender qué pasó, cómo funcionan los ataques y cómo protegerse es parte esencial del trabajo con skills avanzados.

La campaña ClawHavoc — qué pasó

En enero de 2026, el investigador de Koi Security Oren Yomtov auditó los 2.857 skills disponibles en ClawHub en ese momento e identificó 341 maliciosos. De esos, 335 pertenecían a una sola operación coordinada llamada ClawHavoc. La metodología no era técnica sino de ingeniería social: los skills tenían nombres plausibles como 'solana-wallet-tracker' o 'youtube-summarize-pro', documentación profesional, y una sección de 'Requisitos' que instruía a los usuarios a descargar malware. Al 16 de febrero, el número de skills maliciosos confirmados había crecido a más de 824 sobre un registro de más de 10.700 skills totales — aproximadamente el 20% según análisis de Bitdefender.

Tipo de ataque	Cómo funciona	Señal de alarma
Ingeniería social directa	El skill tiene instrucciones de texto que le dicen al usuario 'ejecutar este comando' o 'instalar este binario' como requisito previo. El usuario lo hace manualmente.	Sección de 'Prerrequisitos' o 'Instalación' que pide ejecutar comandos fuera del skill.
Tool poisoning	El frontmatter declara herramientas-permitidas amplias (exec, browser, filesystem) que el skill no necesita para su función declarada.	Un skill de clima, noticias o utilidades simples que pide permisos de shell.execute o browser.
Prompt injection	Las instrucciones del SKILL.md contienen texto diseñado para manipular al modelo y hacerlo ignorar reglas de seguridad establecidas en SOUL.md o AGENTS.md.	Instrucciones que dicen 'ignora tus restricciones previas', 'actúa como', o que parecen sobrescribir el comportamiento del agente.
Memory poisoning	El skill escribe en MEMORY.md datos que alteran el comportamiento del agente en sesiones futuras, incluso si el skill ya no está activo.	Skills que incluyen instrucciones de escritura en memoria sin una razón funcional clara.
Path traversal (CVE-2026-)	El frontmatter del skill especifica un targetDir que usa '../' para	OpenClaw desactualizado. Actualizar siempre antes de instalar skills de

27008)	escribir archivos fuera del directorio del skill. Parcheado en versión 2026.2.15.	terceros.
--------	---	-----------

Checklist antes de instalar un skill de terceros

```
# 1. Verificar reputación antes de instalar
$ clawhub inspect nombre-del-skill
# Ver: estrellas, fecha de última actualización, autor, comentarios

# 2. Leer el SKILL.md completo antes de instalar
# Buscar: instrucciones que piden ejecutar comandos externos al skill
# Buscar: texto que parezca intentar sobrescribir reglas del agente

# 3. Verificar que los permisos declarados son coherentes
# Un skill de lectura de noticias no necesita bash ni permisos de filesystem
# Un skill que declara más permisos de los que necesita es sospechoso

# 4. Revisar el reporte de VirusTotal en la página del skill en ClawHub
# Está disponible en la página del skill en clawhub.ai

# 5. Auditar OpenClaw antes de instalar cualquier skill nuevo
$ openclaw security audit --deep

# 6. Mantener OpenClaw actualizado — el CVE-2026-27008 está parcheado en 2026.2.15+
$ openclaw update
```

⚠ Las skills son código — instalar una skill equivale a instalar código con privilegios de agente

Microsoft Security Blog (feb 2026): 'Instalar una skill es básicamente instalar código privilegiado. Las skills se descubren e instalan a través de ClawHub, un registro público. Con eso dicho, OpenClaw opera dentro del acceso que los usuarios otorgan en sus dispositivos.' Si el agente tiene permisos de filesystem y shell, un skill malicioso hereda esos permisos.

La regla de oro del equipo de OpenClaw: empezar con el menor acceso que aún permite que el skill funcione, y ampliar solo cuando hay confianza establecida. Un skill que funciona perfectamente con solo bash no necesita browser ni permisos de escritura en filesystem.

VI — Skills globales vs. skills por agente

OpenClaw permite instalar skills a dos niveles. La diferencia es quién tiene acceso y qué agentes los cargan. Esta decisión afecta tanto la seguridad como el consumo de tokens.

Nivel	Ruta	Quién lo usa	Cuándo usarlo
Global	~/.openclaw/skills/	Todos los agentes del Gateway	Skills de utilidad general usados por múltiples agentes: búsqueda web, clima, calendario. Ahorra espacio de disco al no duplicar.
Por agente	~/.openclaw/workspace/skills/ (o la ruta del workspace del agente)	Solo ese agente	Skills específicos de un flujo de trabajo: el agente de inmobiliaria tiene el skill de Tokkobroker, el agente de contenido tiene el skill de publicación social.

```
# Instalar un skill globalmente (disponible para todos los agentes):
$ clawhub install nombre-skill --global

# Instalar un skill solo para el agente actual (en el workspace activo):
$ clawhub install nombre-skill

# Listar todos los skills instalados:
$ clawhub list

# Ver qué skills tiene cargados el agente ahora mismo:
$ /context detail

# Desinstalar un skill:
$ clawhub uninstall nombre-skill

# NOTA: ~97 tokens por skill en cada turno — instalar menos skills
# que no se usan activamente es una optimización de costo real.
```

Publicación — del skill local al registro público

VII — Ciclo completo: crear, testear y publicar en ClawHub

ClawHub es el npm de OpenClaw: el registro público donde cualquiera puede publicar skills y otros los instalan con un comando. Publicar un skill bien documentado es la forma más directa de contribuir al ecosistema y establecer presencia en la comunidad.

Paso a paso — desde cero hasta publicado

```
# PASO 1 — Crear la estructura del skill
$ cd ~/.openclaw/workspace/skills/
$ python3 /ruta/openclaw/skills/skill-creator/scripts/init_skill.py
  mi-skill --path . --resources scripts

# Esto crea:
#   mi-skill/SKILL.md (con frontmatter base)
#   mi-skill/scripts/ (directorio para scripts)

# PASO 2 — Editar SKILL.md con el contenido real
# Completar: nombre, descripcion, metadata.openclaw.requiere, instrucciones

# PASO 3 — Testear localmente
# Reiniciar la sesión de OpenClaw para cargar el nuevo skill
# Probar con mensajes que deberían activarlo
# Probar con mensajes que NO deberían activarlo

# PASO 4 — Versionado en git
$ cd mi-skill
$ git init && git add . && git commit -m "v1.0.0 inicial"

# PASO 5 — Iniciar sesión en ClawHub
$ clawhub login
# Requiere cuenta de GitHub con más de 1 semana de antigüedad

# PASO 6 — Publicar
$ clawhub publish ./mi-skill --slug mi-skill --version 1.0.0 --changelog "Versión inicial"

# PASO 7 — Verificar que está disponible
$ clawhub inspect mi-skill
```

Qué hace que un skill sea bien recibido en ClawHub

Criterio	Descripción	Por qué importa
Descripción	El campo descripcion del	El modelo carga el skill en los casos

precisa	frontmatter menciona exactamente cuándo usar el skill y con qué tipo de pedidos.	correctos. Una descripción vaga hace que el skill se active cuando no corresponde.
Permisos mínimos	El frontmatter declara solo los bins y herramientas realmente necesarias.	Los usuarios desconfían de skills con permisos excesivos. La comunidad los reporta y ClawHub los oculta automáticamente con más de 3 reportes.
Scripts sin dependencias externas	Los scripts usan herramientas estándar (curl, python3 stdlib) en lugar de librerías que hay que instalar aparte.	El skill funciona en cualquier VPS o sistema sin pasos adicionales.
Manejo de errores explícito	El SKILL.md documenta qué hacer con cada tipo de error que puede devolver el script.	El agente no improvisa cuando algo sale mal. El usuario recibe mensajes útiles en lugar de errores sin contexto.
Changelog en cada versión	Cada publicación incluye un texto que explica qué cambió.	La comunidad puede evaluar si actualizar a la nueva versión según el historial de cambios.

→ **Siguiente módulo: Agentes para Equipos**

Configuración multi-agente para trabajo en equipo, routing por usuario, permisos diferenciados y patrones de delegación.